Module 2

cocotb Fundamentals

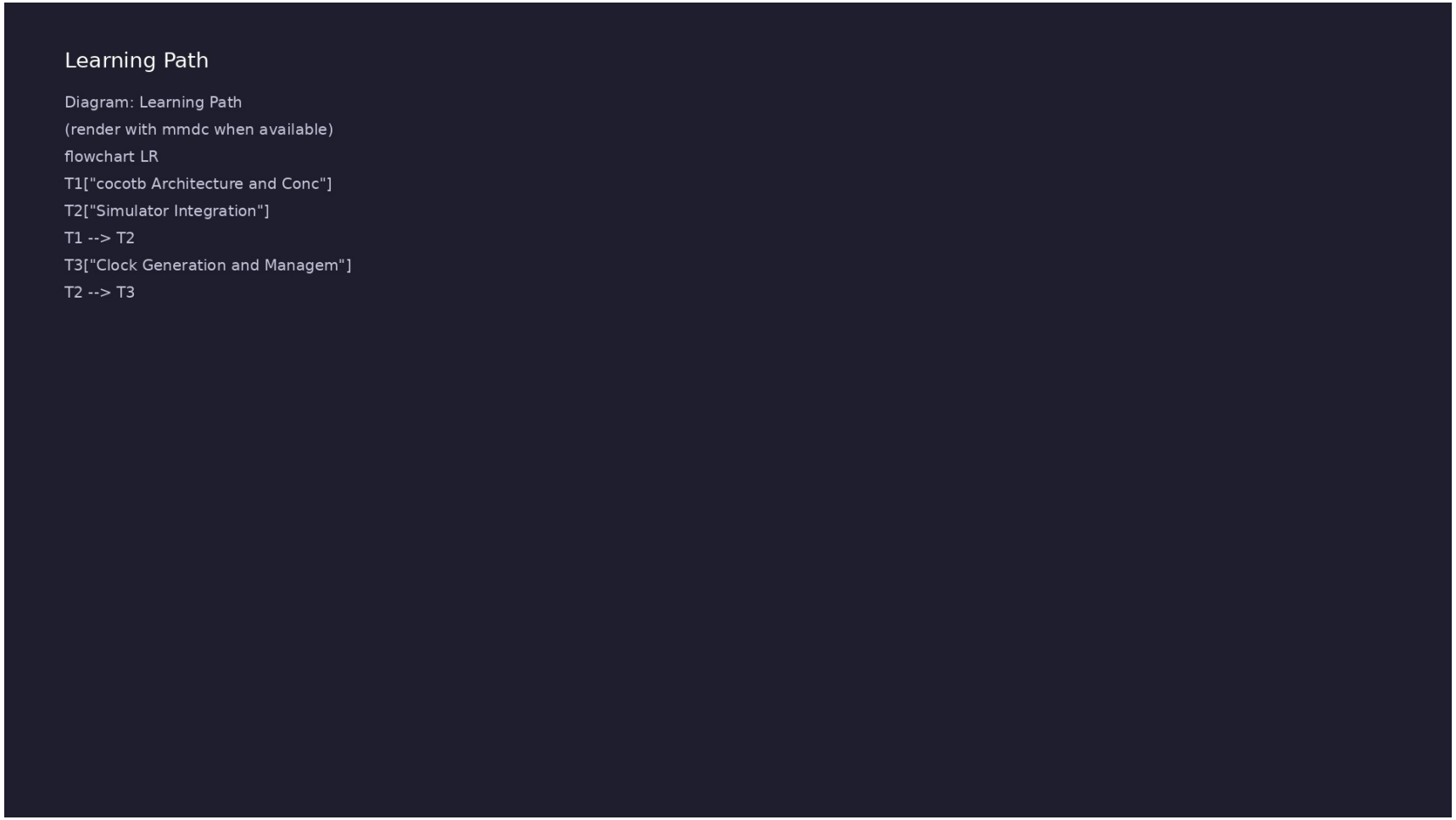
Learning objectives

- Master cocotb for hardware verification

Prerequisites

- See module README

Learning path



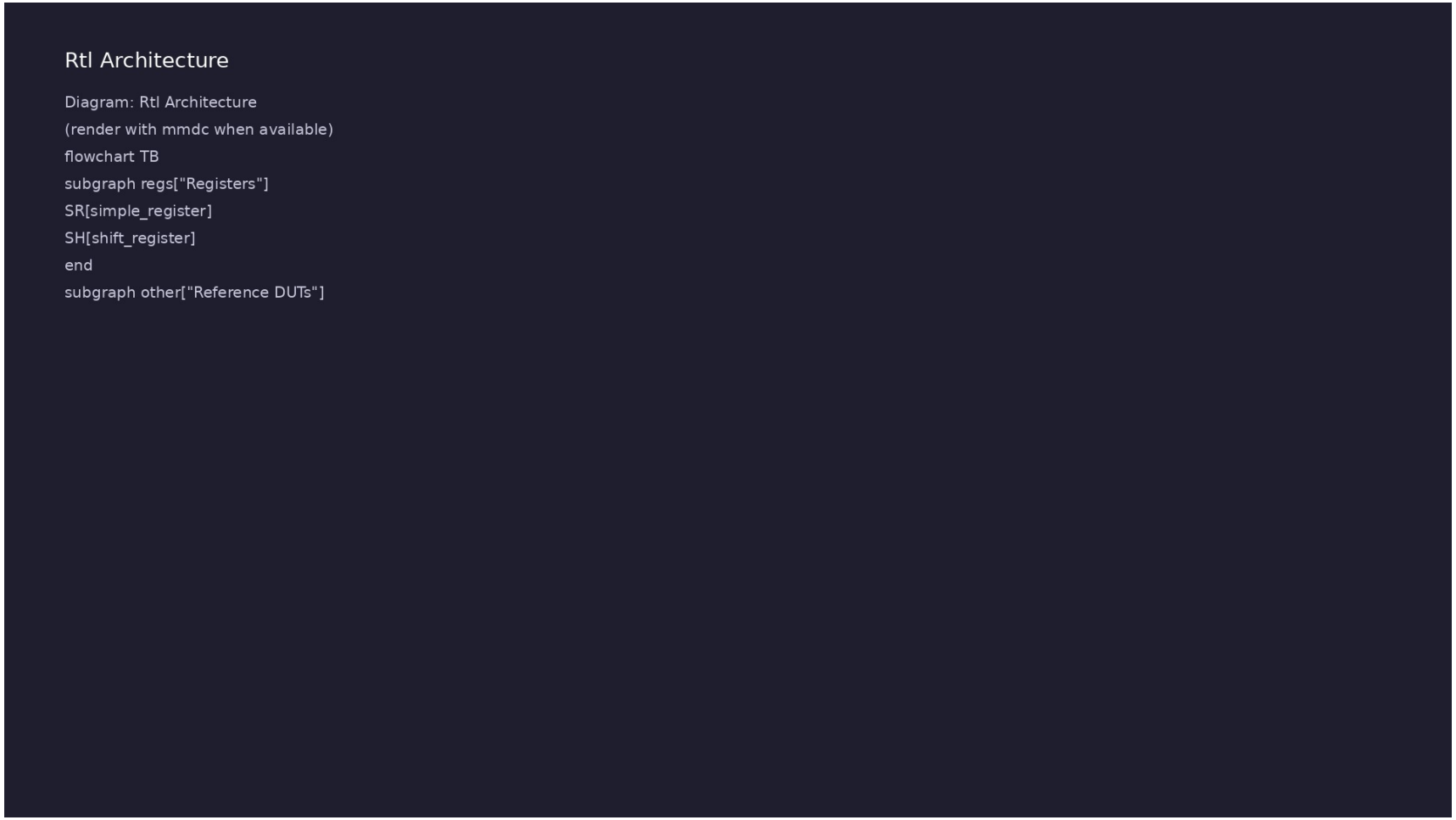
Master cocotb for hardware verification

Overview

- This module provides comprehensive coverage of cocotb, the coroutine-based testbench framework that…

Design architecture

RTL / block architecture



Module 2: DUT structure and key interfaces (see common_dut/rtl/).

Verification environment architecture

Module 2: UVM layers, agents, and checkers for this phase.

1. RTL portfolio architecture

- `simple_register.v` — 8-bit register, enable, synchronous reset (primary lab DUT)
- `shift_register.v` — serial in/out, parallel tap; exercises multi-cycle behavior
- `simple_fifo.v` — 16×8 FIFO with full/empty flags (reference for extension)
- `simple_fsm.v` — IDLE/START/WORK/DONE four-state controller

2. cocotb testbench architecture

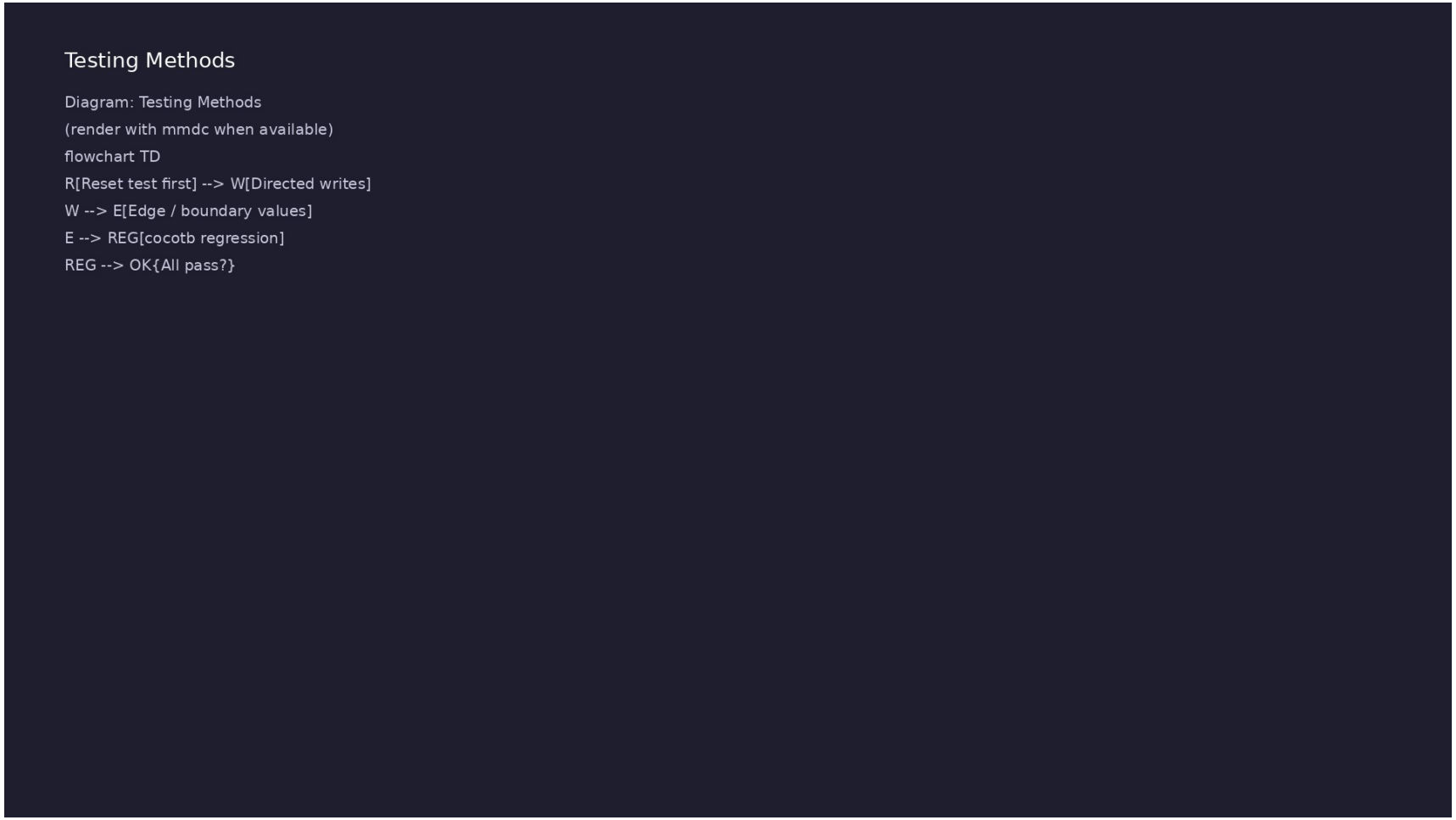
- Test module → coroutines (`@cocotb.test`) → DUT signals via `dut.signal` handles
- Clock generators in examples decouple timing from test logic
- No UVM hierarchy; flat Python functions and shared fixtures via Makefile `TEST=` selection

3. Example vs test separation

- `module2/examples/` — signal access, clocks, triggers, reset patterns (runnable tutorials)
- `module2/tests/cocotb_tests/` — regression tests bound to specific DUTs
- Orchestrator `scripts/module2.sh` routes flags to examples or cocotb makes

Verification & testing methods

Testing and closure flow



Module 2: how tests, checks, and metrics close verification goals.

1. Stimulus and observation

- Directed writes/reads on register ports; shift tests exercise serial timing
- `RisingEdge`/`FallingEdge`/`Timer` triggers structure concurrent activity
- Reset tests run first in regression order to establish known state

2. Regression and Makefile flow

- `make SIM=verilator TEST=test_simple_register`
  selects cocotb test module
- Regression runner executes multiple tests; pass/fail
  per `cocotb.regression`
- Boundary tests (`test_register_all_values`) cover
  0x00, 0x7F, 0x80, 0xFF

3. Debug and closure

- Logging via `cocotb.log`; VCD from simulator flags for waveform debug
- `./scripts/module2.sh --check` validates examples and key cocotb tests
- Assessment: clocks, triggers, reset sequences, structured cocotb tests

Syllabus topics

1. cocotb Architecture and Concepts (1/3)

- What is cocotb?
- Coroutine-based testbench framework
- Python testbenches for Verilog/VHDL
- Simulator abstraction
- History and motivation
- cocotb Architecture

1. cocotb Architecture and Concepts (2/3)

- Python testbench layer
- Simulator interface layer
- DUT interaction
- Event scheduling
- Key Concepts
- Coroutines for simulation

1. cocotb Architecture and Concepts (3/3)

- Triggers for synchronization
- Handles for signal access
- Simulation time management

2. Simulator Integration (1/3)

- Supported Simulators
- Verilator (recommended)
- Icarus Verilog
- ModelSim/QuestaSim
- GHDL (VHDL)
- VCS, Xcelium (commercial)

2. Simulator Integration (2/3)

- Simulator Selection
- Environment variables
- Makefile configuration
- Simulator-specific features
- Compilation Process
- Verilog compilation

2. Simulator Integration (3/3)

- cocotb library compilation
- Linking process
- Makefile structure

3. Clock Generation and Management (1/3)

- Clock Generation
- `Clock` class usage
- Clock parameters (period, units)
- Starting clocks
- Multiple clocks
- Clock Patterns

3. Clock Generation and Management (2/3)

- Regular clocks
- Gated clocks
- Clock division
- Clock stopping
- Clock Domain Management
- Multiple clock domains

3. Clock Generation and Management (3/3)

- Clock domain crossing
- Synchronization between domains

4. Signal Access and Driving (1/4)

- Signal Handles
- Accessing DUT signals
- `dut.signal_name` syntax
- Signal value types
- Signal properties
- Reading Signal Values

4. Signal Access and Driving (2/4)

- `.value` property
- Integer conversion
- Binary representation
- Signal state checking
- Driving Signals
- Assigning values

4. Signal Access and Driving (3/4)

- Integer assignment
- Binary string assignment
- High-impedance (Z) and unknown (X)
- Signal Types
- Single-bit signals
- Multi-bit signals (vectors)

4. Signal Access and Driving (4/4)

- Buses and arrays
- Bidirectional signals

5. Triggers and Coroutines (1/4)

- Trigger Types
- `RisingEdge(signal)`
- `FallingEdge(signal)`
- `Edge(signal)` (any edge)
- `Timer(time, units)`
- `ReadOnly()` (end of time step)

5. Triggers and Coroutines (2/4)

- `ReadWrite()` (during time step)
- `Combine(*triggers)` (multiple triggers)
- `First(*triggers)` (first to occur)
- Coroutine Execution
- Defining coroutines
- Starting coroutines

5. Triggers and Coroutines (3/4)

- `cocotb.start_soon()` vs `await`
- Parallel execution
- Coroutine Synchronization
- Waiting for triggers
- Coordinating multiple coroutines
- Timeout handling

5. Triggers and Coroutines (4/4)

- Exception propagation

6. Test Structure and Organization (1/3)

- Test Function Structure
- `@cocotb.test()` decorator
- Test function signature
- DUT parameter
- Test organization
- Test Phases

6. Test Structure and Organization (2/3)

- Setup phase
- Test execution
- Cleanup phase
- Error handling
- Multiple Tests
- Test discovery

6. Test Structure and Organization (3/3)

- Test selection
- Test parameters
- Test fixtures

7. Reset and Initialization (1/2)

- Reset Strategies
- Synchronous reset
- Asynchronous reset
- Reset sequences
- Reset verification
- Initialization Patterns

7. Reset and Initialization (2/2)

- Signal initialization
- State initialization
- Configuration setup
- Initial conditions

8. Common Verification Patterns (1/3)

- Stimulus Generation
- Sequential patterns
- Random patterns
- Constrained random
- File-based stimulus
- Response Checking

8. Common Verification Patterns (2/3)

- Immediate checking
- Deferred checking
- Reference model comparison
- Scoreboard patterns
- Transaction-Level Modeling
- Transaction classes

8. Common Verification Patterns (3/3)

- Transaction generation
- Transaction execution
- Transaction checking

9. Debugging with cocotb (1/4)

- Logging and Reporting
- cocotb logging
- Log levels
- Log formatting
- Debug messages
- Waveform Generation

9. Debugging with cocotb (2/4)

- VCD file generation
- FST file generation
- Waveform viewing
- Signal tracing
- Interactive Debugging
- Python debugger (pdb)

9. Debugging with cocotb (3/4)

- Breakpoints
- Variable inspection
- Step-through debugging
- Common Issues
- Signal access errors
- Timing issues

9. Debugging with cocotb (4/4)

- Simulation hangs
- Value conversion problems

10. Advanced cocotb Features (1/3)

- Memory Access
- Memory modeling
- Memory initialization
- Memory access patterns
- Bus Functional Models (BFM)
- BFM concepts

10. Advanced cocotb Features (2/3)

- BFM implementation
- Reusable BFMs
- Performance Optimization
- Coroutine efficiency
- Trigger optimization
- Simulation speed

10. Advanced cocotb Features (3/3)

- Memory usage

11. Integration with pytest (1/2)

- pytest Integration
- Using pytest with cocotb
- Test discovery
- Fixtures
- Parametrization
- Test Organization

11. Integration with pytest (2/2)

- Test directory structure
- Test naming conventions
- Test grouping
- Test execution

Hands-on examples

Module 2 self-check

```
$ ./scripts/module2.sh --check
```

Terminal: module2_check

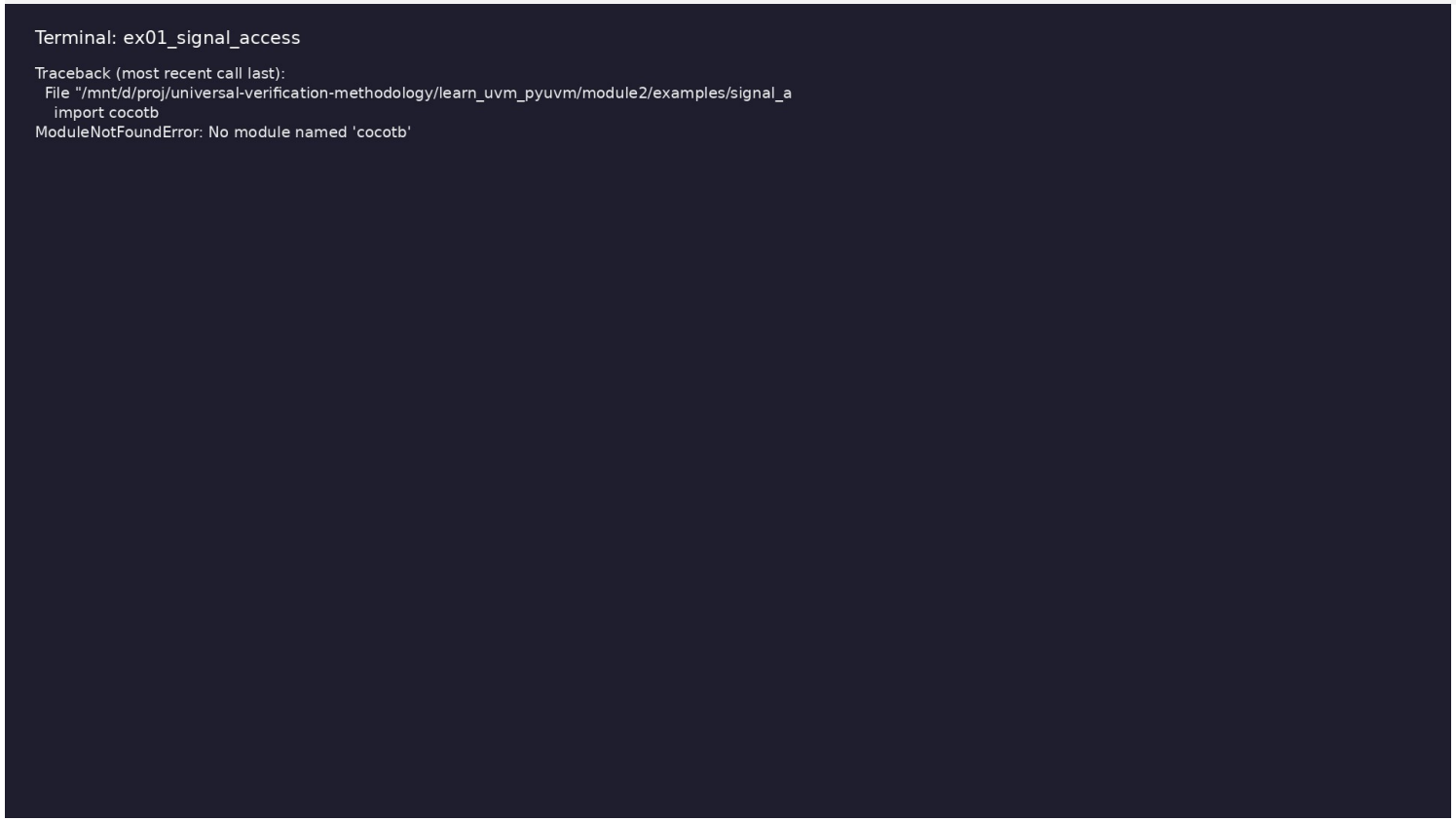
```
[0;36m==========================================[0m
[0;36mModule 2: cocotb Fundamentals[0m
[0;36m==========================================[0m

Usage: ./scripts/module2.sh [OPTIONS]

Module 2: cocotb Fundamentals
This script runs examples and tests for Module 2.

OPTIONS:
    Examples:
        --signal-access       Run signal access examples
        --clock-generation    Run clock generation examples
        --triggers            Run trigger usage examples
        --reset-patterns      Run reset pattern examples
        --common-patterns     Run common verification pattern examples
        --all-examples        Run all examples (default)
        --skip-examples       Skip all examples

    Tests:
        --cocotb-tests        Run cocotb tests

    Environment:
        --venv DIR            Virtual environment directory (default: .venv)
        --no-venv             Don't use virtual environment
        --sim SIMULATOR       Simulator to use (default: verilator)

    Other:
        --help, -h            Show this help message

EXAMPLES:
    # Run all examples
    ./scripts/module2.sh

    # Run specific examples
```

Exercise scaffold

```
./scripts/module2.sh --scaffold
```

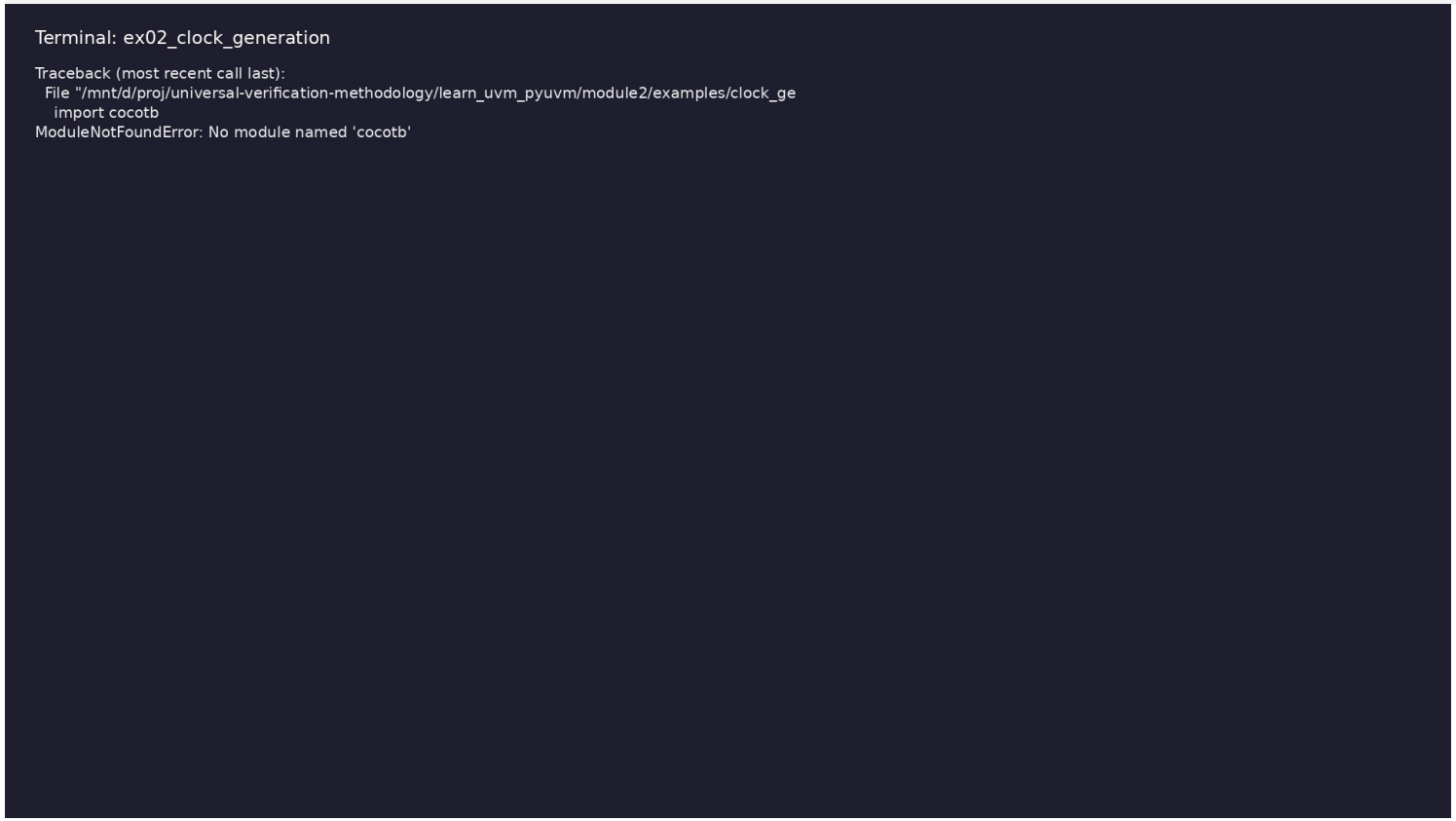
Demo: Signal Access

```
$ .venv/bin/python
      module2/examples/signal_access/signal_access_example.py
```



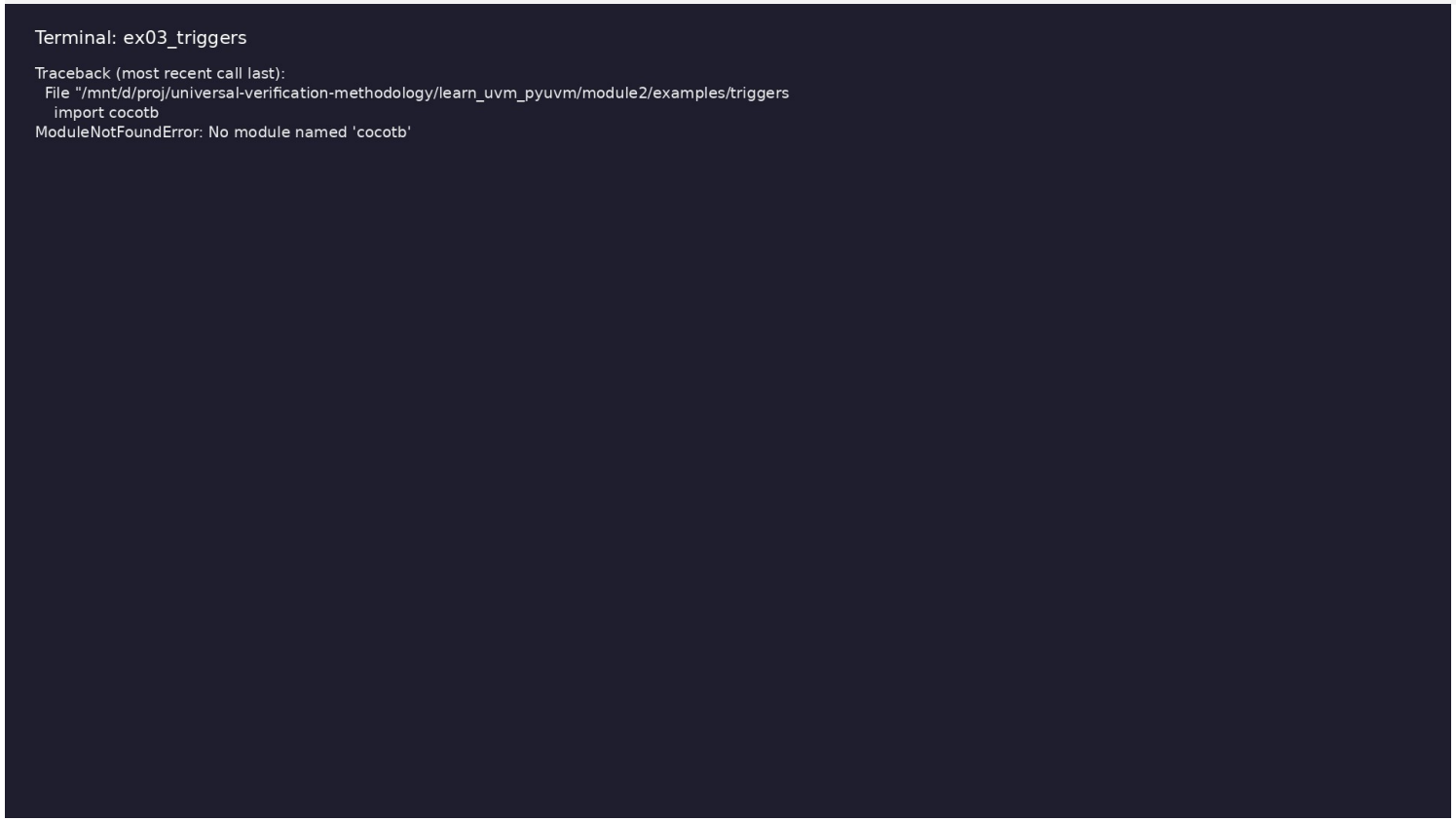
Demo: Clock Generation

```
$ .venv/bin/python
      module2/examples/clock_generation/clock_generation_example.py
```



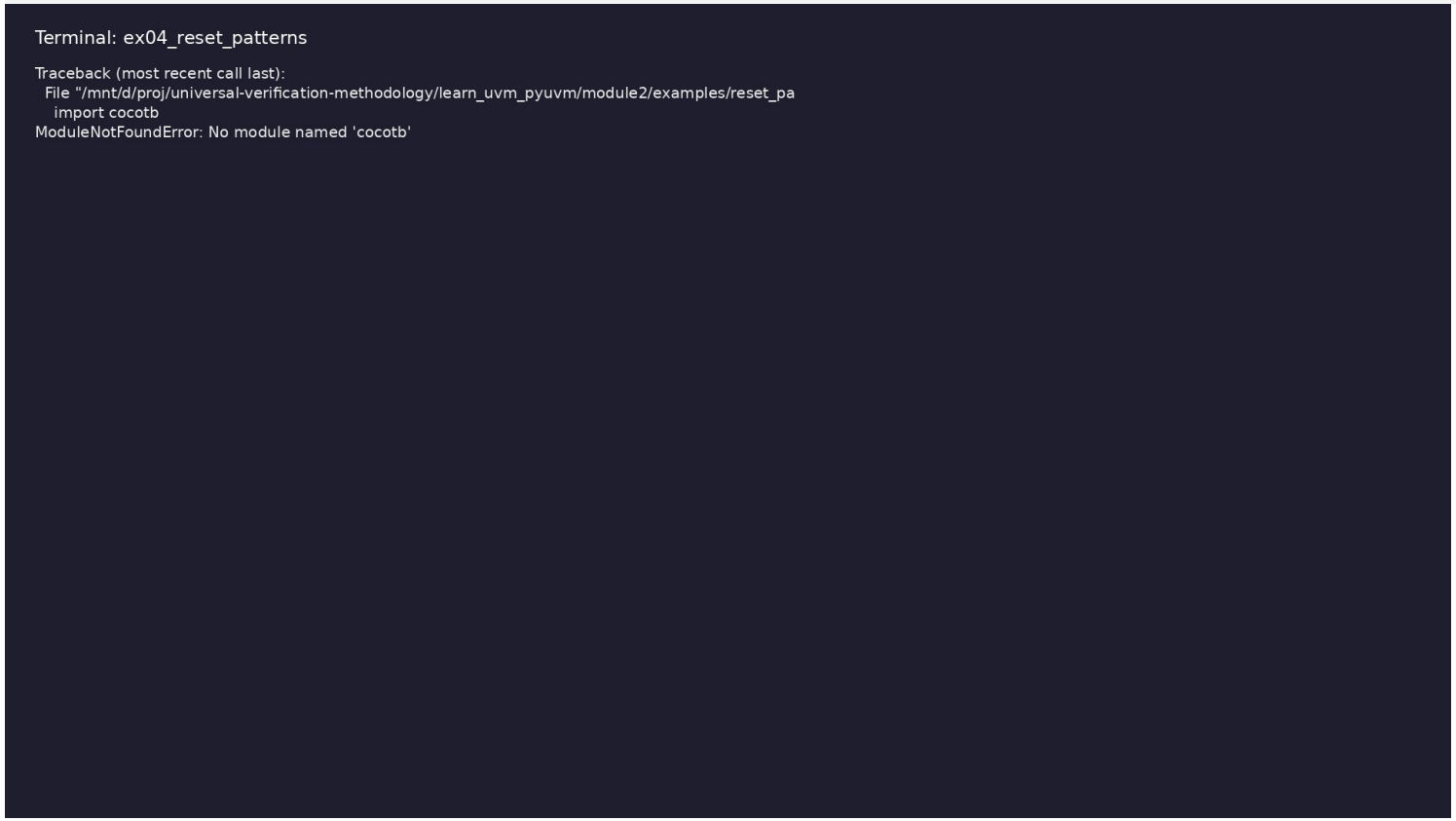
Demo: Triggers

```
$ .venv/bin/python module2/examples/triggers/triggers_example.py
```



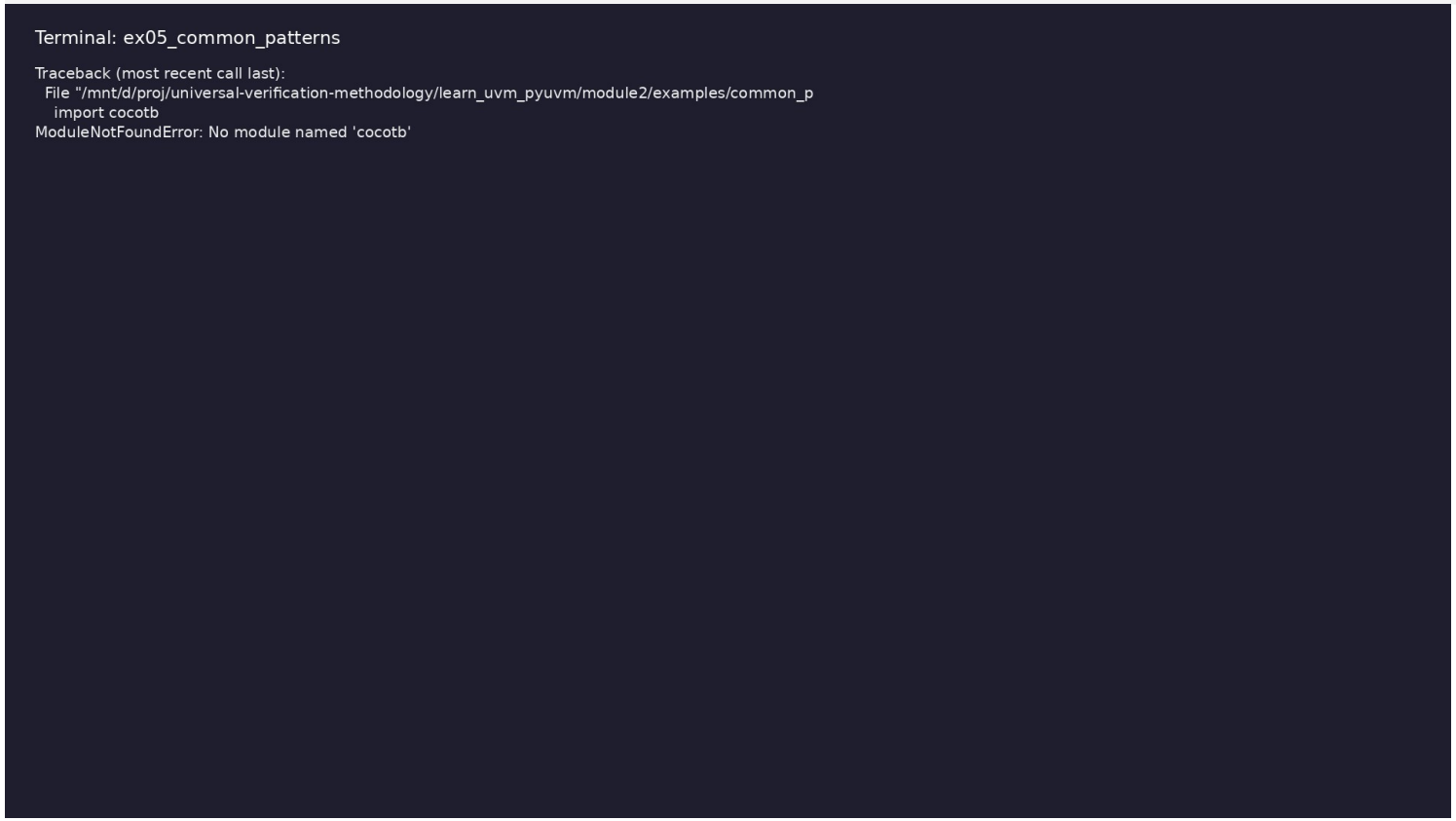
Demo: Reset Patterns

```
$ .venv/bin/python
      module2/examples/reset_patterns/reset_patterns_example.py
```



Demo: Common Verification Patterns

```
$ .venv/bin/python
      module2/examples/common_patterns/common_patterns_example.py
```



Practice & assessment

Exercises

- Clock Management
- Signal Operations
- Trigger Patterns
- Test Structure
- Debugging

Assessment checklist

- Understands cocotb architecture
- Can integrate with simulators
- Can generate and manage clocks
- Can access and drive signals
- Can use triggers effectively
- Can structure tests properly

Summary & next steps

- Master cocotb for hardware verification
- Complete module2/CHECKLIST.md
- Run ./scripts/module2.sh --check
- Next: Next module in course